ActInf ModelStream 016.1: Eli Sennesh and Tommaso Salvatori "Divide-an

ModelStream | 1:14:02

Hello and welcome. This is Active Inference Model Stream 16.1. It's December 6th, 2024.

We're here with Eli Sinesh and Tomaso Salvatore discussing divide and conquer predictive coding.

So thank you both for joining. Eli, to you for the presentation.

All right. Hi, everyone. So I recall being here once before, you know, presenting that old paper, Interoceptionist Modeling Allostasis as Control.

And I think at the time I even mentioned that there wasn't really an inference algorithm that, like, I could just go and apply to all of these kinds of, like, active inference and predictive coding problems.

And over roughly the last year, let's say, you know, we've started work, this whole team of myself, Hao Wu, who is unfortunately not on the stream due to having a nine to five.

And Tomaso Salvatore, who's here with us. You know, we've actually been working on this, you know, like, how would we take these theories and, you know, this predictive coding idea and just turn it into a Bayesian inference algorithm you can use for stuff.

So, you know, as background, like, to really refresh ourselves.

This is the super short, you know, NeurIPS version of our talk.

You know, there's, like, basically theoretical neuroscience, neuro AI, or now, I don't know, Tomaso, like, what keyword do you guys use it versus? I should have asked you maybe.

Or, we do use neuro AI, actually, bioplausible deep learning. That's a sentence I often use.

Yeah, so...

I think that that dates back from even, like, before my versus times, I guess.

Yeah, so the broad theory is that, you know, predictive coding sort of solves two tasks for the brain, and that's credit assignment and Bayesian inference.

Now, like, predictive coding is also a thing in machine learning now.

They do a lot of it at versus. Tomaso's done a bunch with it.

You know, but it doesn't always...

For various reasons we can go into, like, it sometimes doesn't always scale as well as backprop, basically.

And so, we basically started over again, I guess you could say, with the concept of predictive coding to build our divide-and-conquer PC algorithm.

And we basically find that you can use this at roughly the same scale you would with VAEs.

So, of course, in order to, you know, test it out, we had this deep latent Gaussian model.

It's actually the same one from, I think, the Monte Carlo predictive coding paper out this year.

We really did this a lot, like, leaning on, you know, comparing to other predictive coding papers rather than to some arbitrary baseline.

Because, you know, we figured, let's...

If we pitch this as being about bioplausibility, then we can sort of maybe actually get results that are good enough to publish.

Any...

Any...

Yeah.

So, you know, we racked up our first win there.

You know, we went and compared to other algorithms that were related or inspired us.

So, you know, basically we would say, all right, you have...

Yeah.

Oh.

You know, after, you know, demonstrating that this quote-unquote wins, like, okay, it works well enough.

The...

Which...

Oh.

Oh.

This entire slide, yes.

Okay.

Yeah.

So, our DCPC, in addition to being a predictive coding algorithm, is a particle algorithm.

So, it's training a particle cloud, so to speak, to approximate the posterior distribution in a Bayesian inference problem.

So, as such, we compared it to this algorithm particle gradient descent, PGD, from which we were drawing pretty direct inspiration.

And then we also compared it to, you know, Langevin predictive coding out at ICML this year.

And so, we compared on, you know, the pressure inception distance that people use in deep learning, rather than on, you know, the variational bound to the surprise.

You know, we didn't compare on log evidence, because that's, you know, you can often get very bad visual samples from a model that has a very good log evidence, actually.

So, in order to, you know, sort of say, like, okay, that, you know, log evidence is not the only thing that counts, we, you know, compared on this fresh at inception distance.

Over here, we've got some samples from our pseudo-posterior predictive distribution.

So, it's a proper predictive distribution, but constructed from a pseudo-posterior.

Pseudo in the sense that, like, we're fitting, I think, a small Gaussian mixture model to our big particle cloud of the actual posterior distribution.

And then, having done that, we say, all right, now we'll draw samples from this GMM as a pseudo-posterior, you know, then pass those latents through the generative model again to generate samples.

And, you know, the actual neural architecture here is basically a one-level VAE.

Like, you just chop off the encoder from a one-level VAE that would be used in these baseline papers, use the same neural architecture in PyTorch, and then basically just got, you know, slightly better results visually by just improving the environment.

So, you know, the reference algorithm and the training.

Okay, did you manage not to lose that?

Cool.

Okay, so sort of the interesting stuff about it, you know, especially compared to previous work, is how this actually works.

As I said, it's a particle algorithm, so we're actually drawing samples.

I think I said before we went live, you know, this is sort of the year of Monte Carlo predictive coding methods.

We're one of maybe, you know, three or four that are out this year.

So after you've, like, initialized your graphical model by just doing, you know, ancestor sampling to get yourself some initial, you know, cloud,

what you can do during, you know, every inference pass is start from the bottom up.

So that's, you know, the right over here would be the quote-unquote bottom of the hierarchy.

You can calculate, you know, your prediction errors, which are the same kind of thing as in previous predictive coding work.

You know, there's score functions with respect to the latent variable.

And the thing that we actually do differently from other work is that rather than just taking, like, a score function as a local gradient of the, of,

let's say, rather than taking a score function as, like, a direct gradient of the variational free energy,

we take score functions with respect to the log complete conditional distribution.

So, like, a coordinate update distribution.

If you have, you know, this variable Z1, then its complete conditional is the distribution of this variable Z1 given its entire Markov blanket.

So given both, you know, the predictions coming in from above it and the observations coming in from below.

So that's why rather than one, you know, error unit we hear, we have an error unit that connects, you know, Z1 to the observation below it.

We also have an error unit that sort of recurrently tries to draw Z1 back towards its local prior until that prior is changed by an update at Z2.

So if you think about, you know, Bayesian inference works, you know, Bayesian inference is just what you get when you have one distribution and it has to be normalized.

But you're trying to fit it as closely as you can to both the likelihood and the prior at the same time.

And so when you try to do that locally within a big structured graphical model, you know, the logical posterior to go after is the complete conditional.

And that's what we actually did.

And I want to shout out Hal here because he's not on the stream, but his previous work on amortized population Gibbs samplers

is really what showed that this kind of sampling from the complete conditionals could work at scale and, you know, like inspired this project pretty directly.

Like this is kind of follow up work to, you know, his paper on that.

And the way that we, you know, sort of make this work in terms of constructing a free energy

or in terms of making sure we target the correct joint distribution

is that we can use Langevin proposals.

So, you know, if you have a prediction error from E sub zero and another, you know, E down sub one, add them together.
You have the gradient of the log complete conditional.
Now it's the log unnormalized complete conditional, but that's fine because, you know, the gradient doesn't, like, you know, when you take a gradient of a log density, the normalizing constant falls out.
And so you can target that with, you know, a Langevin dynamics proposal,
which just says take your previous prediction,
you know, multiply your prediction error by a learning rate,
add the prediction error to the previous prediction, and then add some Gaussian noise.
You know, you have to balance, like, the,
you have to balance the magnitude of the Gaussian noise with the learning rate in a certain way,
but, like, that's a detail.
Really, it's saying let's be in the latent space,
then let's follow the gradient, and then let's inject a little bit of noise
so we explore most of the time.
Or rather, sorry, so we explore some of the time while exploiting
whenever the gradient signal is strong enough.
And then, you know, I note that we use sequential Monte Carlo here,
and what that's actually for is that sequential Monte Carlo algorithms
give you a whole set of rules for how you can start from,
you know, the proposal that you sample from,
then go to the local target that you are actually trying to,
you know, that you're trying to sample from,
and then combine those in, you know, a neat way to get
samples from the complete joint distribution.
So you would do this, you know, start here from the data on the right,
do this predictive coding step for Z1,
be done with it,
repeat the same predictive coding step for Z2,
but now with a new prediction error from Z1 down here,
repeat all the way up here,
and then you have a bunch of samples from different complete conditionals,
and you have to come up with some way to say,
well, what's my free energy?
Or in SMC terms, what's my importance weight?
And that's really where, you know,
that's really where, like, the SMC technique becomes helpful,
is for proving the whole thing is correct.

So if you think, you know, we named it divide and conquer,
because, you know, what we're essentially doing
is we divide the graphical model
into these complete conditional units,
then we conquer each of them
with, you know, basically a Monte Carlo predictive coding proposal,
and then we conquer the whole thing by recombining them.
So it really is like a divide and conquer algorithm
in the classic sense.
Also, the reviewers wanted us to name it something
clearer than the old name.
All right, so then, you know,
that's a summary of the talk, really,
and then we can go into, like, details of the paper
and, you know, anything else we want to talk about.
But broadly, the idea here is that, you know,
predictive coding as a theory, you know,
enjoys biological plausibility
for approximate Bayesian inference,
which is, you know, actually this really hard task.
Like, you know, in deep learning world,
people use all of these awkward neural network
arrangements to try and amortize inference.
And then the people who are real purists
are just doing Markov chain Monte Carlo
because amortized inference isn't good enough for them.
And to be fair, it's often not very good
if you don't, you know,
structure your amortized posterior,
like, just so.
And more to the point,
in neuroscience, you know,
like, predictive coding theories
have often been, you know,
built on, like, mean field assumptions
about the approximate posterior.
So then you just have a coordinate-wise representation
of the approximate posterior.

And it's pretty well known
that, like, this can't actually get you
a very close approximation
to a correlated structured posterior.
So that was where the trouble was.
That was the problem that we took a first step on solving.
You know, and we got it to work well enough
that you can substitute it for neural networks
in some of these cases
where people use neural networks.
And we're going to be at the poster session
next Friday at NeurIPS
if anyone wants to come and talk to us.
You know, and of course,
we have to mention the places that we work
and our affiliations.
Awesome.
Thank you.
Thank you.
I'll stop the screen share.
Yeah, I'll ask some questions.
And if anyone in the live chat has questions,
they can write.
Maybe just starting with the structured part.
What do you mean that
it's a structured problem?
Oh, yeah.
So, like, this is the experiment,
you know, originally,
I looked at this and said,
well, how do we actually replicate
house results from advertised population Gibbs?
And that was like a full-on time series model
in which there were, you know,
like some local variables
that were updated at each time step,
but also some global random variables
that were standing, you know,

at the hierarchical level above.
So, you know, that was really,
like, I thought,
okay, we need to tackle
these kinds of structured problems.
Because, you know, if you,
I mean, go look at,
I'm trying to think, like,
you know, honestly, most of the time,
most interesting generative models
are going to be kind of structured.
If there's more than one latent variable,
and it's not just IID sampling,
so any kind of hierarchical bays,
you know, time series structure,
dynamical system structure,
like, all of this stuff that you find
in cognitive science and neuroscience
so, so commonly,
and yet, which is often,
you know, we've often sort of
given up on doing anything
better than particle filtering
or amortized inference
if you look in the machine learning world.
Okay, and the graphical structure
you showed was a linear hierarchical model,
or it was going from side to side,
but that's just graphical convention.
So, do you ever need to account
for branching
or any other kinds of structures,
or how do you divide and conquer
when there's also splitting and merging?
Okay.
So, I mentioned about Markov blankets, right?
As long as you're in a directed
acyclic graphical model,

then there's still a notion
of a Markov blanket
for every single random variable.
That's the thing that you divide,
you know, the graphical model into,
is variables with their Markov blankets.
And, you know, this is sort of
intended to be a bit free energy principle.
Perhaps bringing the notebook up
would help.
I'll ask the two questions,
so return to them as you see fit.
The first is,
how do you specify the model?
What does it look like before training?
And what does get updated during training?
And the second question is,
how do you take that step as a modeler
between the structure of a given problem,
like the handwriting digits,
and adapting that to what?
Is that something more generic?
Or does that graph need to already,
from some other source,
have structure that's amenable to the target?
So the target density
that we would try to sample from
is defined by the graphical model structure.
And like that, yeah,
that has to come from somewhere.
So conventionally,
you know,
if you're doing machine learning engineering,
then that's sort of up to your choice
as the user of our algorithm.
You know,
it depends on what,
like,

domain knowledge you have
about your application.
And if we were talking about brains,
then that structure of the generative model
is something that evolved.
You know,
and there could be,
you know,
and I should mention,
you know,
the recent versus work,
like,
of course,
you can also have,
you know,
structure learning problems,
you know,
throughout,
like,
development,
but,
you know,
throughout development
and through certain behavioral tasks.
But like,
that gets you a bit beyond the realm
of just graphical models.
And so,
you know,
we,
we basically aimed at
just a given graphical model.
Take a graphical model
as given
and try to do inference in it.
You can work your way up
to the harder stuff later.
In practice,

we often use deep neural networks
because we,
we try to replicate
the experiments
of other papers.
So we had like multi layers
and the convolutional models.
Yeah,
maybe share the notebook
and or the repo.
Working out.
Yeah.
Oh,
the GitHub repo is like,
yeah,
we can share that pretty easily.
Yeah.
I'm going to put this
first in the Zoom chat,
then share my screen.
Yeah.
So in the GitHub repo,
we try,
we attempt.
Just zoom in a little bit.
That's actually like,
sure.
Yeah.
Yeah.
So in the GitHub repo,
minus a requirements file,
we try to share enough
that you could hypothetically
download and run this
like just from your command line.
You would need,
you know,
a Python environment

that's got the necessary stuff installed,
but all of that is standard stuff.
So if you wanted to just rerun
the MNIST experiment,
you would literally run this command
on your command line.
And then you could also,
you know,
start your tensor board
and watch it train.
You know,
then we include,
like,
evaluation notebooks
that,
you know,
basically would help you
to replicate the figures
from the paper.
And,
yeah,
we actually have
yeah,
we also share,
you know,
some pre-trained,
like,
weights and particles.
I know that's
probably a bit
of a colliding name
with,
like,
weights and biases.
In this case,
you know,
the weights include the biases.
Weights would mean

neural network weights
and then particles
would mean,
you know,
the frozen samples
from the posterior distribution
for random variables.
And then,
of course,
we just give,
you know,
the paper,
or sorry,
the table of results.
And as a small bonus
to everyone else
who has to try
and replicate our results,
we include
an implementation
of the discretized
Gaussian distribution,
which is otherwise,
like,
buried in the
diffusion models literature.
Thank you very much.
Yeah,
and it's,
you know,
under MIT license,
so, you know,
you can pretty much
go ahead and,
like,
fork this,
contribute that.
It's in the

universal probabilistic
programming language,
Pyro,
since that's basically,
like,
the largest
deep probabilistic
programming language.
Let's take a look
at some of the notebooks.
Let's see.
That one's
going to look
like garbage.
This one's
going to look
like not garbage.
Yeah,
so,
what this does
is,
you know,
it's basically,
like,
PyTorch
and PyTorch
Lightning,
you know,
so you're going to,
like,
read some JSON
file,
and that's going
to give you,
you know,
your data,
your model,
and your trainer.

Then,
you know,
you're going to remind
the trainer
to just load
the weights
and particles
from a checkpoint.
You know,
the random variables
are saved as part
of the PyTorch
Lightning
and checkpoints.
Yeah.
It loads them,
blah,
blah,
blah,
blah,
blah.
Right there,
when it says
357,000 trainable
parameters,
what is that
number
coming from?
That is the number
of neural network
weight.
Like,
that's the number
of float 64s
in the neural
network weights.
So that's actually
a relatively small

number because
this was sort
of a toy problem,
and normally
it would be
more in the
millions.
And where
were those
combinatorics
specified?
Right here
in the
architecture
specification.
That's after
it,
though.
Yeah,
because you have
to load the
thing to print
the thing.
Okay,
okay,
that's just
printing it out.
But you specified
it elsewhere,
what the structure
would actually be,
loaded it in,
printed it out.
Okay.
Yeah,
the JSON
file,
you know,

it tells you
like which
modules in the
Python code
that you have
to load
and all of
that.
So if we
go look at,
you know,
what is the
actual training
file that we
were loading
here,
it tells you,
you know,
load this
model,
module,
pass it these
hyperparams,
checkpoint this
often,
you know,
load this
data set
with like
this hyperparam
parameter and
batch size.
You know,
here's like
learning rates
and the
number of
particles,

you know,
the number
of sweeps,
meaning like
passes over
the graphical
model,
you know,
that you do
per training
step.
Yeah,
this is like
relatively standard
deep learning
type stuff.
yeah,
and when we've
loaded the
architecture,
we actually
print it out.
You know,
so it's a
simple like
isotropic
Gaussian prior,
and then we're
running it
through,
you know,
a linear
layer,
an activation
function,
you know,
a
deconvolution

layer,
some batch,
you know,
the,
you know,
each layer
includes
batch
normalization,
you know,
two more
convolutional
layers with
their batch
normalization
and nonlinearity,
two more
convolutional
layers again,
and then
finally a
deconvolutional
layer that
gives you the
correct image
size and
channel number.
well,
that's rude.
Anyway,
yeah,
so then
we have
this method
we call
load
particles,
which just

tells you
that's really
for loading
the particles
into GPU
memory,
once you
know which
data items
you're testing
on,
again,
like relatively
standard,
you know,
deep learning
stuff,
you know,
then here
we say
we're going
to open
a Python
context
handler
by,
you know,
clamping
the graphical
model
to the
pre-trained
particles
that we
loaded
from disk.
disk.
So,

you know,
if you
load
your
posterior
particle
cloud
from disk,
then you
have to
say to
the graphical
model,
I'm testing
right now,
don't do
any additional
inference
steps,
just use
these
samples,
you know,
as your
distribution.
So,
then we
run,
we average
over
particles
to get
our
reconstructions
of,
you know,
training
images.

Now we're
going to
plot them.
Here's the
actual
reconstructions,
so like
they're,
you know,
kind of blurry
like you expect
from a VAE,
but it
shows that,
you know,
things did
actually work.
Then you
say,
okay,
I don't
want to
look at
reconstructions
of training
images,
you know,
I want to
know,
do I
have a
trained
generative
model?
So you
clear all
that stuff
out,

and you're
going to
get all
of the
training
and validation,
you know,
particles
from
disk
for
training
your,
you know,
for fitting
your pseudoposterior,
and you're
going to
predict some
new images,
you know,
using the
pseudoposterior.
Then,
lo and
behold,
you know,
you plot
out the
samples from
the pseudoposterior.
They're kind
of blurry,
and they're
not as
varied as
you would
sort of

like them
to be,
but that
is what
happens when
you have
a very
simple
generative
model.
Like,
you know,
if you go
back to this
neural
architecture
here,
this is
nothing like
a state-of-the-art
diffusion model
or whatever
that you would
use for image
generation.
It's really
just a bunch
of neural
layers stuck
together,
mapping a
latent
variable to
an observation.
variation.
Yeah.
And then
we're going

to,
you know,
having
generated
those,
we go
through,
we calculate,
you know,
some metrics
across the
validation data
under a
series of
different random
seeds.
We can also
sample from
the prior
rather than
the pseudoposterior,
and that
gives us
lower visual
quality but
more variation,
more realistic
variation.
And then,
you know,
we calculate
the statistics
that we reported
in the paper.
Plus some
statistics we
don't report
in the paper

now that I
think of it.
Like the
effective
sample size,
that one's a
good one for
inference nerds.
You know,
you divide that
by the number
of particles
you're using
and it gives
you a sort
of percentage
that tells
you,
you know,
how well
are you
actually fitting
the true
posterior
distribution
with this
batch of
samples.
And in
our case,
the answer
is,
yeah,
about 92%
effective
samples.
Thanks.
Maybe

could we
walk through
a similar
logic for
the MNIST
handwriting?
Yeah,
totally.
Let's see.
I think it
would be
almost equivalent.
It's just
that we use
a different
model.
Yeah,
it's pretty
much the
same,
but with,
you'll notice
there's
quote-unquote
three decoders
here.
So each
of these
is actually
performing,
you know,
a random
Gaussian sampling
step
in between.
So this
is a
proper

hierarchical
Bayesian
problem.
So you
have a
prior,
then you
have a
forward
conditional
and a
forward
conditional.
So you
have three
levels of
latent
variables,
and then
finally you
have your
likelihood.
What
leads you
to use
those
here?
Like,
do you
sweep
or scan
over
two,
three,
four layers?
Do you
explore out
features equals

other
multipliers?
Or what
led you to
this
specification
for the
Lightning
DCPC?
So
in this
case,
we took
this exact
specification
from
Monte Carlo
predictive
coding,
which is
over
here.
So we
wanted to
compare
inference
algorithms
head to
head.
So we
picked
exactly the
same
generative
model
structure
and
replicated

it.
Awesome.
Yeah,
that makes
a lot of
sense and
it helps
with the
continuity
and
isolating
the
inference
algorithmic
component.
is there
methods
or
procedural
step
anywhere
other
than
these
or
all
that
are
needed,
these
JSON
configs
and the
notebooks
like this?
So this
is runnable,
like this

should be
runnable
if you
actually
just
download
it and
follow the
links to
download the
pre-trained
weights,
put them
in the
path that
the notebook
expects
here,
then you
should just
be able to
run this.
It should
be point
and click.
I say
should
because
you know,
like I've
tried it,
but I've
tried it
once.
Cool.
Definitely
opportunities for
people to

explore.
And then what
about the
training
step?
So that
would be
done from
the command
line as
the repo
readme
tells us.
that's
this step.
And you
watch on
TensorBoard
and make
a manual
call based
on diagnostics
or how
else do
you determine?
I mean,
I would
generally say
let it run,
at least for
these experiments,
let it run
to completion.
Like,
you know,
I know that
if I'm
stopping

something
partway in
because of
the diagnostics
I see on
TensorFlow,
you know,
like for
instance,
a very
noisy,
staticky,
like,
loss curve,
well,
that's how
I know
when it's
not working.
You know,
the experiments
that are
mentioned in
the paper
are actually,
you know,
known to
work.
Like,
you can just
leave that
running until
it's done
and you
will get,
you know,
subject to
differences in

your random
number generator
more or
less the
result be
got in
the paper.
I actually
want to say,
like,
this is one
of the nice
things about
the,
like,
sort of the
culture of
machine learning
these days,
is that
everything is
really expected
to be
not just
replicable,
but,
like,
idiot-proof
replicable.
As in,
if you
cannot just
run this
by rote
and get
the same
result,
then reviewers

don't believe
the result
is real.
Which is,
you know,
very much for,
like,
replicability
of science
the way
it actually
should be.
So we
looked at
faces and
digits.
Let's say
somebody were
looking at
another kind
of image
classification
problem,
cats and
dogs.
so what
would need
to be
adapted?
What kind
of data
would be
in for
the training
step?
And then
to what
extent do

these same
Lightning
DCPC
specs,
structures
work?
Or to
what extent
do the
first or
the secondary
layers need
to be
adapted to
alternative
cases?
So this
thing that
gets called
a graph
inside the
Lightning
DCPC,
you'd have
to write
a new
one of
these
classes
for a
new
problem.
So let's
say,
for instance,
that you
want to
do

semi-supervised
Bayesian
learning.
My old
advisor from
my PhD
had a
paper about
that.
So if
you're doing,
say,
cats or
dogs,
then you're
going to
want to
specify a
new graphical
structure that's
rooted in a
prior
latent
embedding.
And then it
should have
a pair of
likelihoods,
essentially,
one being to
the class
label,
which is
going to
be semi-supervised.
So let's
say you only
get class

labels on
about half
the samples.
And the
other is
going to
be for the
actual image,
which is going
to be fully
observed.
after writing
a
high-torch
lightning data
module for
your data
set,
which
tells you
when,
which,
sorry,
it handles
figuring out
which data
points are
supervised
versus
unsupervised,
because it's a
semi-supervised
problem.
And then,
yeah, you would
put that
together and
run it more

or less,
you know, you
would write
another little
JSON file and
just run it.
And hopefully
you would get,
hopefully, if
your, you know,
architecture for the
generative model is
rich enough,
then your latent
embedding space
should begin to
capture, you
know, differences
between the
supervised classes
and be, you
know, and give
you good prediction
on classification.
And then if you
wanted to do
classification-style
testing or
evaluation,
you know, you
would present a
whole new image
as the
observation
and then just
perform joint
Bayesian inference
on both the

latent embedding
and the
class label
and then check
that your class
labels are
good, so to
speak.
You know, for an
unobserved class
label, you're not
getting a one-hot
vector, you're
getting, you
know, a series
of logits or
like an element
on the simplex
that specifies
probabilities of
class membership.
so hence my
saying good
rather than
absolutely
correct.
You never get a
completely absolute
yes-no answer out
of a classifier
at test time,
at least.
I think in
general, you could
also generalize it
to different kinds
of data, of data
types.

So it does not
have to be, for
example, continuous
data as images.
You could be, you
could generate
discrete distribution
distribution or you
could also generate
time series
distribution.
This is something
that has not been
tested in this
paper, but in
theory, the
theoretical framework
would allow to
do so.
Yeah, like we
have an appendix in
the paper that
extends the math
to discrete
distributions.
You know, that
does take a
mathematical
extension and
then that, you
know, would
take some more
coding.
So like we
don't, you
know, make any
promises there.
So in, in

past dreams and
journeys, Eli, as
you mentioned, we
discussed with
Dean and others
the anticipatory
allostasis.
And then
we discussed the
causal modeling.
So where, where
would you
triangulate this
work?
Or how do you
see it as sort
of drawing
on or
complimenting or
not?
But it's just
interesting that
it's, it's not a
homeostatic model.
Often in the
organismal,
ethological active
inference, there's a
lot of focus on
organismal behavior
in these kinds of
topics, homeostasis,
allostasis being
critical there.
And then in the
Bayesian modeling
space, often
there's a focus on

the interpretability
of smaller
models and
carving nature at
the joints and
all of that, not
this more
machine learning
sort of specify
it in a
sequence of
convolutional
layers.
So how do you
see all, just,
just having worked
on those different
areas, how do
each of you see,
I mean, because
clearly this is a
relevant contribution
and advance.
So in what
direction or how do
you relate it with
those prior pieces?
I think it depends
in which direction
you want to, you
want to push this
work a little bit
forward.
So for example, if
we're looking at the
machine learning
side and we want to
scale it up to

larger models, larger convolutional networks, you don't get interpretability because of this method.
I think the same interpretability you would have with the old algorithms, you have them with this one.
What you gain apparently, what this work shows, is that you get better performance.
But then of course, if you apply to, for example, directed graphical models or Bayesian networks that have a specific causal structure that you can observe, in that case, you're able to state, you can do, for example, the intervention kind queries that I presented here like a year ago or something like that.
But I think that's not in the algorithm, it's more in the kind of

model you pick up.
Yeah, it's the kind
of task that you
want to solve.
And whether you
care about it or
not.
So let's say maybe
for generating
phases for now, we
did not care about
having an interpretable
model.
What would you
say?
Yeah, Eli.
I would sort of
compare it to
something like a
car engine.
You know, you
need to design a
car engine of a
certain size and
power in order to
put in a certain,
you know,
in order to put in
a certain like weight
class of vehicle.
But that doesn't
actually determine,
you know, whether
that vehicle is a
Honda or a Jaguar.
If it's the same
size and, you know,
weight class, then you

can actually use a
very similar engine
inside both of
them.
And I think that's
sort of on the one
hand, the both the
beauty and the pain
in the ass of working
with Bayesian
methods, is that,
you know, the
promise on the
sticker is that
here's Bayesian
methods and you're
going to solve a
very broad range of
problems with, by
reducing them to the
same problem, which
is Bayesian
inference.
And that's, you
know, sort of what
the free energy
principle is all
about, is saying
it's all really just
reducible to
variational inference,
right?
But now what you
do find, if you go
look at like Pi
MDP or a lot of,
you know, free
energy type papers,

is that they'll
design a model for
the paper in order
to model some piece
of interesting
organismic behavior,
but they'll include
some real
restrictions on the
expressivity of that
model, which is that
it has to be
solvable by something
like, say,
variational message
passing.
So then there's
the question of,
well, how do you
stop restricting
the model class
according to what
inference algorithm
you're going to
use?
And that was really
like the dirty
secret of Bayesian
methods that I
learned in grad
school was you're
always restricting
the model class
to what you have a
good inference
algorithm for.
Or at absolutely
last resort,

you know, running
something like
plain old
Metropolis Hastings
with a Gaussian
random walk
proposal and
just waiting for
like three weeks
of compute and
hoping.
So, you know,
what we actually
want to do here
is to say we
have a better
engine.
Like, literally,
we have a better
inference engine.
You don't have
to, you know,
now structure an
entire neural
network architecture
of inference
proposals.
You know, you
don't have to
play graduate
student descent
with your
inference algorithm
too much.
You know,
let's, assuming
that, you know,
brains have

some kind of
wondrous generic
inference algorithm
that solves so
many different
Bayesian
problems all at
once.
you know,
let's try to
model that and
then just use
it because it'll
be more powerful
and more efficient
than, you know,
customizing to the
particular generative
model you want.
and, you know,
really it's like
first baby steps
but it is
encouraging that,
you know,
if you,
I think we show
in the table,
yes, like we
trained a
variational auto
encoder, you
know, as a
baseline to say,
well, what if
you're doing
variational inference
by actually, you

know, training a
neural network to
do it for you
and it's
encouraging to
show that we
can do a
little bit better
with like a
generic inference
algorithm that
doesn't need a
separate neural
architecture, you
know, compared
to like build a
second neural
network and
train it jointly
with the first
one.
I think
something that
is also
exciting about
this, I mean,
at least for me,
is that we,
like, we've
been, we've
been working a
lot with
predictive coding
algorithms for
machine learning
in, and I
guess you can
clearly separate

them in like
supervised learning
and unsupervised
learning.
And in
supervised learning,
for example,
you train like
large convolutional
models on C410,
C4100, or
timing image net,
it seems that
we've kind of
like hit a
momentary wall.
So it's, up to a
certain point, it's
becoming really hard
to scale up those
algorithms.
things.
And I think like the
whole community, and
it's not only
predictive coding, like
when I talk with, for
example, the
equilibrium propagation
pre people with the
bio plausible deep
learning community,
there's a kind of
similar problem that
is when the model is
more than seven,
eight layers deep, it
doesn't perform well.

And for example,
scaling it up to
image net is a
pain.
So a lot of people are
taking a step back and
say, okay, let's do a
little bit more research
to find out why and
generalize and try to
improve the performance.
When using, for
example, predictive
coding for generative
AI, so to generate
images to sample
data points in an
unsupervised way, the
first problem that we
had is that like in
machine learning, we
had a couple of
until like two years
ago, it was a
mostly a deterministic
algorithm.
So we were taking
mostly Rao and
Ballard's version,
which was the
classification one,
there was no
stochasticity.
So a bunch of
people, including me,
started publishing
like associative memory
papers.

It's like I give it
a data set and I
can, by giving it a
little bit of
information, you can
regenerate exactly the
same data points.
And then in the last
year and a half, as
I said, some people
started developing
sampling algorithms
and they've been
basically the papers
before hours a little
bit, either they do
it on MNIST and
fashion MNIST or
they do it on like
colored images like
C410, C4100, but
they use some kind
of gradient descent.
So it's not a
completely local
message passing
algorithm.
And what we did in
our paper is that we
have a local message
passing algorithm and
we compare against
theirs by using
basically exactly the
same architectures,
exactly the same
number of epochs.
I think in some cases

even exactly the same
learning rate and
things like that.
message passing
algorithm.
And as a small
correction, we aren't
actually a message
passing algorithm.
Yes.
We're not technically
any form of some
product message
passing.
No, that's a local
algorithm, let's say.
Yes.
A local algorithm,
yes.
And we do well.
Like, for example, the
images of faces you've
seen may not seem
perfect, but they are,
at least from my
perspective, better than
expected.
And the thing is, we
haven't, I don't, I
don't feel like we have
reached a wall in terms
of performance.
Like we've tested only
on the architectures that
other people have used
before and we do well.
And for now there's a
little knowledge on how

much this can, this can
scale up.
Maybe you scale it up
like two, yeah, two
layers more and doesn't
work anymore.
That's something that we
don't know yet.
But potentially it would
be interesting to, for
example, something I was
messaging with Eli
earlier this week is, can
we scale it up to large,
larger decoder only
transformer models, for
example, not like of
course, LLMs or things
like that, but for
example, something that
is, uh, like a four or
five, six, uh, layers, the
transformer model that
would already be an, uh,
like a quite an interesting
result.
Definitely not an easy one,
but I think there is, there
is definitely room to
improve what's in the, the
results of this paper.
Could you also maybe
highlight how, how do you
see local update rules as
superset subset, what have
you of message passing?
What differentiates the
local locality of

connectivity from something
that you would explicitly
Eli call a message as
passed.
Ah, that's an interesting
question.
I guess is, uh, I think
for message passing, you, you
could call like all the, uh,
belief propagation
algorithms, or at least
basically some kind of this
class of algorithms where you
clearly state which
information has to be
passed.
uh, from, uh, from
basically one latent
variable to the other.
While, uh, local message
passing and, and there are,
in those kinds of
algorithms, they're all
local.
So it's, uh, since, since
you define them to be this
way, they, they, they must
have local information.
Well, probably if you
generalize to local message
passing in general, there are
some proposals, maybe
including this one in which
maybe things are not
supposed to be local.
Like maybe you can like,
since it's a structured
learning algorithm, you may

in theory get information
from other, from light and
variables that are more far
away, but is a, but then
when you actually do the
computation, they are local,
but that's a, that's an
answer I'm giving now to the
spot.
Maybe like as a better thought
on this.
So I would say that message
passing algorithms differ from
ours in both their means and
their goal.
And really it's the goals
actually that they're starting
from.
So that family of algorithms,
you know, when, when people
were working on it, maybe, I
don't know, 20 years ago, what
they were really trying to do was
to get a summary of a posterior
marginal distribution.
So they just wanted to be able to
pick out one latent variable for
an inference query or a decision
they needed to make.
Say, what is the posterior on this
variable, given the data, not given
any settings for all the other latent
variables?
variables.
And that sort of restricts, you
know, the space of tasks that you
can really address with message
passing.

And it also, you know, does end up
meaning that, you know, what you're
basically doing is some form of some
product or really integral product, you
know, algorithm.
And that gets you all the way out to,
I think, probabilistic circuits are the
most current and advanced form of some
product based, like exact inference
algorithm.
And they only support tree shaped, you
know, generative models rather than
those that are, you know, directed
acyclic graphs in general.
So trees rather than forests.
And we said, well, let's try targeting,
you know, the full joint distribution of
all the latent variables together.
Partly because that's the most general
form of evasion inference problem.
And partially because, you know, if you
have something like a hierarchical time
series model from the active inference
literature, you might need several levels
of the hierarchy.
You might need the latent variables
across those levels in order to make,
you know, an action decision.
You know, several levels of hierarchy
might actually parameterize your policy
or the thing that defines the inference
problem you care about solving, you know,
as a modeler.
That's very interesting.
Interesting.
Again, to the sort of settings that a lot of
active inference literature has tackled.
In these cases, it's like digit in pixel intensities, etc.

Digit classification, generativity of digits out.
How would it be similar or different with something like digit pixels in,
TMA's behavior out, or robot arm out?
Are you leveraging this identity or at least structural congruence
with the recognition and the generativity?
Or like a VAE, can you have something different coming in and have the output be something that's
categorically different than the type that was used as input?
Like action.
So I think one of the advantages of defining anything as a Bayesian inference problem
is that it really changes your conception of what's an input versus an output variable.
You know, for a Bayesian inference problem, you start with a graphical model.
You clamp several variables to be observed data.
Those observed data are now your inputs.
You know, your output is any aspect of the posterior distribution that you care to examine.
So we don't have to be outputting, you know, classification logits.
We could also be outputting, you know,
yeah, we could be outputting, you know, functions.
You know, you could have, say, an active inference time series module
where your agent sort of gets some data
at every step of computation,
performs a full sweep over the entire, you know, history of latent variables,
and then and only then actually,
you know, makes a decision about what action it's going to take.
Which really highlights the importance of being able to specify the full joint distribution
and then clamp down with nuance
on what you want to be taking as observable,
but kind of clamping down from the full joint
and then just specifying and then having an inference algorithm
that will facilitate that locally, operationally,
rather than trying to
amidst the constraints of which inference procedures you can perform,
work within subclasses of graphical models that abide by those constraints.
It's almost like it's a far cry in some ways from 2022 textbook,
figure 4.3,
the kind of POMDPs
and the matrix multiplication
that gets implemented in PyMDP,
the sorts of pedagogical cases

that do highlight certain key moves.
this, though,
looks and feels so much more like a
machine learning work,
and
it's
really interesting, though,
what this enables.
I think you also shape the work
with respect to the venue you want to submit to.
Yes, I was just going to say,
now that I've found a figure 4.3.
When we were discussing which experiments to run,
the first question is,
are we going to submit a new RIPs?
Yes.
It's going to be okay.
Then it's going to be
mostly image generation.

also a little bit the venue
and a little bit what did prior work do?
Right.
The image generation.
We picked out image generation
because
predictive coding networks
had
not yet been used
to get,
you know,
this quality of result
on unconditional image generation.
So what,
other than
the conference next week,
what ways
might you be

taking it?
I think maybe
the,
I mean,
probably there are many future directions.
I guess
from the,
from the perspective
that interests me the most,
I would be really curious
about doing what,
what I mentioned earlier
about scaling it up.
So for example,
what's the,
what's the,
how much can we push
this kind of algorithm?
So it's,
I think at some point
if we're going to have enough compute
and maybe a faster,
it is,
this is like
some fast training,
but probably like
if it's implemented
in a,
in a parallelizable library
or things like that
is a,
you can run
larger models,
you can run
a huge number
of hyper parameter search
and,
and you can see

at some point
where it scales up.
Cause it's also true
like we didn't do,
I mean,
yeah,
there is a,
like we didn't do
a huge hyper parameter
search at the end.
Like we need some,
but it's not,
but like definitely
not comparable
to what standard
in a machine learning paper
in which they try
like thousands
of combination
of hyper parameters.
No,
we did not do
grid sweeps
of like hyper parameters.
Yeah,
yeah,
exactly.
So,
so I think there's,
there's room
for pushing this
for sure.
We have a little bit more
and may hopefully
much more,
but this is a,
I think scalability
is something

that is really hard
to,
to intuitively predict.
Like why some models
work well on seven layers
and then on 10
they collapse
is a,
you just,
you just do it
and notice that.
And then I guess
you can do some
interpretability afterwards
and understand the reason
but doing it before
is always a,
a tricky thing.
And so I guess
trying it is probably the,
yeah,
if you have a method
that allows you
to try fast
and fail fast
is the best way
of scaling it up,
I guess.
Yeah.
So I would also say,
you know,
based on some
of the not so
publishable results
that I've seen
in,
for instance,
you know,

trying to replicate
how's work
with like bouncing
MNIST digits,
you know,
how well this works
often does seem
to depend
on exactly
how much
structure
the generative
model provides
you,
you know,
to give like
prediction errors
that point
in the right direction
rather than,
well,
in exactly
the right direction
rather than just
vaguely
the right direction.
Right?
So,
you know,
what is sort of
how smooth
versus how peaked,
tightly peaked
is the landscape
of your,
like,
is your free energy
landscape.

And when it's
more tightly peaked,
this algorithm
seems to work,
you know,
significantly better
versus when it's
not so tightly peaked,
it'll often
sort of get
into the region
of valid solutions
and then get
very noisy
as it becomes,
like,
unable to decide
which of the
potentially valid
solutions
is really
the best one
to converge on.
You know,
I would say
it's not so good
yet at dealing
with, like,
multimodal
posterior distributions
once a good
generative model
has been learned.
So they're scaling
it up to that.
You know,
I'm also interested
to see, like,

how far could we
take this on a
fairly standard
machine learning
problem
by just saying,
you know,
every time we have
more layers
than would be
biologically plausible
to differentiate,
just insert
a random variable.
Like,
parameterize a
Gaussian right there
and then just say,
okay,
now,
you know,
the amount of
gradient calculation
you have to do
between layers
is biologically
plausible
and we'll use
the predictive
coding math
to actually,
you know,
effectively solve,
like,
the,
we'll have
the random
variables substitute

for backpropagation
in a certain way.
What do you
mean by that?
So,
if you have,
like,
a,
you know,
12-layer,
I think,
Tommaso,
you said,
like,
a 12-layer
energy-based
transformer
or something?
So,
I think
the general
concept about
biologically
plausibility
is that
you have a latent
variable,
another latent
variable,
and you want
anyone that
is basically
computing the
gradients of the
parameters to
depend on both
latent variables
and not on

other stuff that
are in between.
So,
for example,
if you consider
the function that
goes from one
latent variable
to the first,
from the first to
the second,
to be a three-layer
network,
the gradients of
the weights of
the intermediate
layer or of the
first,
they would depend
on the last latent
variable,
and this would
basically make you
break the
locality assumption
and biological
plausibility.
While if you
consider the
function to be a
single feedforward
or convolutional
layers,
it means that for
every weight,
you have a latent
variable before it,
so you have a

presynaptic neuron
and a postsynaptic
neuron actually
in practice,
and this will
basically allow
you to,
once you compute
the gradients
of the free
energy,
to have everything
that depends on
presynaptic and
postsynaptic neurons.
So this is the
concept with
biological
plausibility.
The problem
that, for example,
I'm noticing
when you do
classification
is that
doing this
allows you to
propagate the
error a little
bit less
regularly than
backprop does,
because backprop
is sequential,
so it shines a
little bit for
very deep models,
especially if you

have the right
activation function.
So I guess
that's what
Eli was
mentioning when
he was saying
about...
Yeah, so if
you have, for
instance,
12...
If you have,
for instance,
image, 12
layers,
classification,
right,
then,
roughly speaking,
you know,
getting a gradient
for the activations
of the middle
layer of the
neural network
depends on
all the way
on the final
observation.
Now, what
we'd like to
say is,
well, how
much, you
know,
ultimately,
like, you

know, it's
been proved
that predictive
coding in the
limit implements
backprop just
very slowly,
but what we'd
kind of like to
do is to
calculate gradients
a bit more
quickly, and so
what I was saying
is that, you
know, what you
could do is
sort of take
the middle
layer and
just say,
well, this is
going to be a
random variable
now.
Like, instead
of, you
know, a single
deterministic
activation that
has to be
exactly right
in order to
pass gradients
through, this
is now going
to be a
Gaussian

distribution
over activations.
And having
done that, you
can then use,
you know, our
predictive coding
algorithm to
say, I'll
try and, you
know, propagate
through the
latent space
of possible,
you know,
weights, or
actually, no,
sorry,
I'll try to
propagate through
the latent
space of, like,
layer activations,
find, you
know, a
relatively good
solution, or
rather a, you
know, particle
cloud of
relatively good
solutions for
those, and
then update the
weights in
order to make,
you know, those
activations more

likely without
depending on the
final classification.
So without
depending on the
entire, like,
backwards
computation tape
of backcrop.
very, but at
the cost of
being a little
bit random,
let's say.
What's wrong
with that?
well, nothing
say probabilistic
machine learners
and say free
energy principle
people and,
you know,
active inference
people, but,
you know,
currently,
currently the
machine learning
world doesn't like
it very much.
They say Bayes
doesn't scale.
We're going to
teach them they're
wrong about that.
well, it's
super cool.

If you have
any other
things you want
to add,
otherwise, good
luck with the
conference and
with proceeding.
It's super
exciting, and I
hope people
watching this
go to the
repo,
continue on
some of the
explorations that
you've mentioned
and explore it
in their own
ways, too.
Yeah, I mean,
feel free to send
questions to our
email addresses that
are listed on the
paper if you are
not able to get
the repo to
work.
Or if you
make the repo
work and you
have ideas for
follow-ups.
Yes, that too.
Yeah, we've got a
few ideas already.

Like, you know,
I think I was
talking to Tomaso
and saying, like,
I've been meaning
to write some
code that will
allow you to
sort of pass a
deterministic output
between random
variables so that
we would, you
know, not just be
able to do, like,
graphical models,
but proper
stochastic
computation graphs.
Divide and
contribute.
Exactly.
Yes.
Cool.
Thank you,
fellows.
Until next time.
Thank you very
much, Daniel.
for organizing
this.
Yeah.
Yeah, thanks for
running, Lise.
Like, every time a
new one comes out,
it goes on my
watch later list,

which admittedly I
never work through,
but they do go on
it, and sometimes,
sometimes, one or
two videos.
Cool.
Cool.
When I get the
opportunity to binge,
you are what I'm
binging.
Thank you.
See you guys.
Bye.
Have a nice day.
Bye-bye.
Bye-bye.

Thank you.